Movie Title Autocomplete

Comparing a Trie and a Ternary Search Tree on 150,000 IMDb movies

Administrative

Team Name: Movie Search (Group 123)

Team Members: Aidan Quach (aidanq06), Hana Elbatouty (hana4324), Krish Gupta (KrishGupta25)

GitHub Repo: https://github.com/aidanq06/movie-title-autocomplete

Video Demo: https://www.youtube.com/watch?v=XtHltlB4EbI

Extended and Refined Proposal

Problem

For this project, we built a movie search program that suggests titles after a person enters the beginning of a movie name. The program answers every search with two data structures we built from scratch. The question our group wanted to answer is which structure handles prefix search better in the case where the collection holds a large number of movies, which is 150,000 movies.

Motivation

Autocomplete can seem simple but in order for it to work, a large collection must be searched through in a way that's fast enough to feel instant. A Trie is the typical structure for prefix search and a Ternary Search Tree is seen as a leaner alternative. We wanted to compare their performance on a large dataset to see how they differ.

Features Implemented

We built everything into a terminal program in C++17. When the program starts, the user gets a menu with six options. The user can choose to search using the Trie or the TST or run the same prefix through both at the same time. The other options include starting the full benchmark, pulling up the dataset and index statistics, or quitting. A search gives back a maximum of ten movies, which each has its original title, release year, and IMDb ID. If a movie has no year in the dataset, Unknown year is printed instead. The comparison is how we catch mistakes. It gives one prefix to both structures and checks if they agree with eachother. The benchmark measures construction time, query time, node visits and estimated memory, and can export per query measurements to a CSV file. All result lists come back in the same order from both structures and are checked against an independent sorted scan.

Description of Data

Our data comes from the title.basics.tsv, which IMDb publishes. We chose not to load the whole file into memory. Instead, the loader reads it line by line. It only keeps a row when titleType says movie and isAdult is 0. It quits once it gets 150,000 rows, which happened once it searched 270,772 rows. For each movie, we store four fields, which are the IMDb ID, primary title, release year, and title type. Titles are normalized for searching so Spider-Man Homecoming becomes spider man homecoming and the original spelling is kept for display. The selection has

136,509 distinct normalized titles and 8,976 titles are shared by more than one movie. Keys have an average of 17.4 characters. The years span all the way from 1894 to 2026 and 86 movies have no year. The 1.1 GB file changes daily so our runs are pinned to a dated download with a recorded checksum.

Tools, Languages and Libraries

We used C++17 with the standard library only. CMake builds the project and CTest runs the tests. We utilized different GitHub features like GitHub issues, branches and pull requests to efficiently work together on this project. Timing uses std::chrono and the benchmark workload comes from a seeded mt19937 generator to make sure every run asks the same queries. We did not use any library implementation of either search structure.

Algorithms Implemented

Trie: We used one node per distinct prefix. Each node keeps its child edges in a small sorted vector and finds a child with binary search. This saves a lot of memory compared to using a fixed array that has 37 child slots per node. This would use about 426 MiB at our scale. A search walks one node per character of the prefix and then walks the subtree in alphabetical order until it reaches ten results.

Ternary Search Tree. Every node holds one character and three links called left, middle and right. A matching character follows the middle link, while a smaller or larger character follows the left or right link. It behaves like a small binary search tree built into the structure itself. The TST ends up with the same number of nodes as the Trie apart from the empty root. This means the two differ in node layout rather than node count. Both structures store their nodes in one big vector and refer to eachother with integer indexes instead of by using pointers. Both traverse iteratively with an explicit stack so a long title cannot overflow the call stack. Movies live in one shared record vector and terminal nodes store indexes into it.

Additional Data Structures and Algorithms

We decided to utilize vectors for node storage and results, binary search inside Trie nodes, an explicit stack for depth first traversal, std::stable_sort with one shared comparator for duplicate titles. Furthermore, we used a plain sorted scan that works independently to check for correctness and a set that removes duplicate queries when the benchmark workload is generated.

Distribution of Responsibility and Roles

Aidan Quach built the foundation. He worked on the shared interfaces, the IMDb loader, title normalization, the Trie and the first command line interface and tests. Hana Elbatouty built the Ternary Search Tree and wrote this report. Krish Gupta built the benchmark, the interactive menu and the integration tests and wrote the analysis notes. Each member contributed roughly fifteen hours overall to this project.

Analysis

Changes After the Proposal

The problem context that we aimed to investigate did not change. However, we expanded upon how we implemented it in a few ways. Firstly, the proposal promised at least 100 test prefixes. The final benchmark runs 130 queries split into six categories so we can see where each structure wins instead of just looking at everything through one average. We added an independent sorted scan check. If we just compared the two structures only against each other, we would miss a bug they share. This is why we decided to also compare them against a plain scan that uses neither structure. We benchmarked with a limit of ten and again with no limit. The reasoning behind this is the limit of ten matches what a user sees while the no limit runs expose the cost of walking large subtrees. Our Trie uses sorted child vectors instead of the standard fixed child array. This was a memory decision and it ended up helping our speed result. The interface ended up having additional features compared to the proposal sketch. We implemented a side by side comparison option, a statistics view and CSV export of benchmark measurements.

Worst Case Time Complexity

We will now list the variables and assumptions used for the time complexity. L is the length of a key or prefix. A is the alphabet size which is at most 37 here. S is the number of nodes in the subtree we walk. R is the number of results returned. N is the number of titles. T is the number of file rows.

Function	**Trie**	**TST**
Insert one title	O (L × A)	O (L × A)
Locate a prefix	O (L log A)	O (L × A)
Collect matches	O (S + R)	O (S + R)
Full search call	O (L log A + S + R)	O (L × A + S + R)
Build from N titles	O (N × L × A)	O (N × L × A)
Sort k duplicates at a terminal	O (k log k)	O (k log k)
Load and filter the dataset	O (T × L)	O (T × L)

The worst case for Trie Insertion is O(L × A). Every new edge gets inserted into a sorted vector. In the worst case, that vector already holds A entries that all need to shift over. The TST's worst case comes from walking up to A sibling links per character. A is small and bounded here so both behave close to O(L) per lookup in addition to the size of what they return. Normalizing a query is linear in its length and is paid equally by both structures.

What the Benchmark Showed

The results didn't show what we expected to happen. We expected the Trie to answer queries faster because that is the standard textbook claim. In our implementation the TST won. TST query medians were about 25 to 37 percent lower on GCC and Clang builds. The TST also built faster at about 230 ms against about 370 ms and used slightly less estimated memory at 113.17 MiB compared to 119.00 MiB. Both structures returned identical results for every query on every platform and had zero mismatches. Both also had almost exactly the same number of nodes, which were 1,395,440 and 1,395,441. The table below shows one Windows GCC benchmark run

by query category. The searches were capped at ten results and the times are measured in microseconds.

Category	Mean results	Trie µs	TST µs
short prefixes	10.0	6.35	4.23
medium prefixes	7.6	4.65	3.34
long prefixes	5.7	3.69	2.68
common words	10.0	6.30	4.10
rare prefixes	1.7	1.87	1.51
no match	0.0	0.64	0.70

Our best explanation for TST winning is the memory layout. Each Trie step follows a pointer to a separately allocated child vector which likely costs an extra cache miss. A TST step has its three links inside the node it already read. The TST was faster even though it visited more nodes per query, which was 75.8 compared to 66.9 on average. This shows a node visit is not a fixed unit of work. The no match row is the only place the Trie came ahead because those queries are just descent where its fewer visits matter most. This does not mean TSTs beat Tries in general. An array based Trie would likely query faster and pay far more memory and we did not measure one.

Testing

Six test suites run under CTest and together they make over 900,000 checks when the real dataset is present. They make sure to cover many things from duplicates, missing years, punctuation, empty and nonexistent prefixes, the ten result limit, rebuilding, and tests on 150,000 generated records and separate randomized comparisons against a sorted scan. Everything passed on four toolchains across Windows and Linux. They also passed under address and undefined behavior sanitizers. We planted six deliberate TST bugs and confirmed the tests caught every one.

Reflection

Overall Experience

We believe that the project went well because we split it into multiple parts instead of trying to work together on everything at the same time. The first phase had the foundation, which included the data loader, normalizer, trie, and the shared interface both structures would use. We finished the foundation and the Trie first. The second phase then added the TST, the benchmark, and the menu. Because both structures answered the same calls on the same records, it was easy for us to swap and compare them.

Challenges

Fair benchmarking was the part our team found the most difficult. Times moved by up to 20 percent between runs and the gap between the structures changed with the compiler and the standard library. We repeated every run on four toolchains and trusted the median values over means. Getting identical results out of two different structures also took a good amount of work.

It's only true if both share one normalization function, one traversal order and one comparator for duplicate titles. This meant we had to write tests that prove a deliberately broken TST gets caught.

What We Would Change

We would try to do our benchmark earlier instead of near the end. This is because late timing surprises are hard to investigate. When we saw that the Trie came out slower than we expected, we didn't have as much of a chance to look into why that happened. We would also build a third index, a Trie with a fixed child array. Our cache explanation for why the TST won was just a hypothesis, but using the Trie with a fixed child array would allow us to test it directly. Also, we would measure real process memory instead of estimating from node sizes.

What Each Member Learned

Aidan got experience learning how to design a shared interface that others can build on without leading to any errors. He had to design it so the TST and benchmark could be built on top of his work without anything breaking. Hana learned how a TST works and that memory layout can end up being more important than node count, which is likely why the TST was faster. Krish learned how difficult it can be to benchmark things fairly. He also learned how much testing you have to go through before a measurement can be trusted.

References

[1] IMDb Non Commercial Datasets.

https://developer.imdb.com/non-commercial-datasets/

[2] R. Sedgewick and K. Wayne. Algorithms, 4th edition, Section 5.2 Tries.

https://algs4.cs.princeton.edu/52trie/

[3] cppreference, std::chrono library. https://en.cppreference.com/w/cpp/chrono